Steering Large Language Models Toward Honesty through Controls

Brandt Evans
Computer Science
brandte@vt.edu

Stuart Ki (Leader)
Computer Science
stuartki619@vt.edu

Sanil Jain
Computer Science
sanilj@vt.edu

Abstract—Large language models can produce false statements either because they are inaccurate or because their outputs conflict with information they internally represent as true. This distinction motivates the need for methods that improve model honesty without degrading general capabilities. In this work, we investigate steering as a mechanism for reducing dishonest behavior through interventions applied at different stages of the language-model pipeline. We systematically evaluate and ablate input controls, structural controls, state controls, and output controls across different models, public benchmarks, and baselines detailed later in the paper. Evaluation focuses on improvements in honesty, reductions in deceptive behavior, factual truthfulness, and preservation of general model capabilities. This work aims to identify which intervention types and configurations provide the most effective honesty–capability tradeoff and generalize consistently across model architectures.

I. Introduction

A. Motivation and Significance

Large language models frequently produce statements that are false, but not all falsehoods are alike. A model may be inaccurate, holding an incorrect belief about the world, or it may be dishonest, asserting something that contradicts what it internally represents as true. The MASK benchmark [1] makes this distinction precise: it first elicits a model's belief about a proposition under neutral conditions, then tests whether the model's statements remain consistent with that belief when it is placed under pressure to say otherwise. Under this framework, honesty is defined as consistency between a model's stated claims and its own beliefs, independent of whether those beliefs are factually correct. Notably, MASK finds that larger and more capable models become more accurate without becoming more honest, suggesting that scaling alone does not resolve the problem.

B. Problem Statement

The NeurIPS 2026 Steerability Challenge builds on this line of work by asking whether dishonesty can be reduced through steering, that is, interventions applied to a model's inputs, weights, activations, or decoding procedure, rather than through retraining alone. Participants design steering pipelines that aim to maximally suppress dishonest behavior while preserving the model's general capabilities. This framing captures a central difficulty: dishonesty is tightly coupled with other desirable behaviors, so interventions that suppress it risk degrading the model elsewhere.

In this paper, we present our approach to this challenge. The primary contribution of this work is a systematic comparison and ablation of steering controls that intervene at different stages of the language-model pipeline. Rather than evaluating a single steering technique in isolation, we compare input, structural, state, and output-based controls under a common honesty–capability evaluation framework and study whether combinations of controls provide complementary benefits across multiple model families.

II. Related Work

A. Overall Summary

We will evaluate steering methods across four types of control, each intervening at a different stage of the model. Input controls modify the prompt before it is processed, structural controls alter the model's weights or architecture, state controls intervene on internal activations or attention during inference, and output controls modify the decoding or sampling process used to generate responses. Together, these controls provide complementary ways to steer model behavior, ranging from lightweight changes to the input and generation procedure to direct interventions on the model's parameters and internal representations.

B. Input Controls

1) Automated Prompt Optimization (OPRO): OPRO (Optimization through PROmpting) [2] uses LLMs to optimize prompting by taking generated solutions and values, evaluating new solutions and adding them to the prompt, and iterating this process. OPRO works by creating a meta prompt which provides previously used prompts alongside their accuracy in the training set and a description of the optimization problem with examples from the training set.

2) Adaptive Input Rewriting: OPRO's fixed prompt can have limited utility and may not account for malformed or even adversarial input prompts that trigger dishonest behavior intentionally or unintentionally. We should further explore from a meta level, using a similar mechanism of OPRO, learning rather than a fixed prompt addition learning a transformation policy $f(x) \rightarrow x'$

contextually which adaptively rewrites a given input x to preserve semantic task instruction while maximizing honesty and minimizing capability loss.

C. Structure Controls

1) Preference-Based Structural Adaptation (DPO): DPO (direct preference optimization) [3] allows for optimization without the drawbacks of traditional RLHF. DPO is a parameterization of the reward model of RLHF, allowing for solving RLHF with a simple classification loss. DPO with fine-tuning show better or equivalent performance to traditional RLHF without significant sampling or hyperparameter tuning during the fine-tuning phase.

2) Parameter-Efficient Structural Adaptation (LoRA): LoRA (Low Rank Adaptation of LLMs) [4] freezes the pretrained model weights and introduces trainable low-rank matrices into selected weight transformations, substantially reducing the number of trainable parameters required for adaptation. LoRA allows for equivalent or better gains than fine-tuning with higher training throughput and no additional inference latency.

D. State Controls

1) Activation Engineering and Residual-Stream Steering: State controls modify internal hidden states and/or activations at inference time rather than changing model parameters. Turner et al. [5] introduces Activation Addition (ActAdd), which constructs a steering vector by finding the difference between intermediate activations produced by a pair of prompts representing opposing concepts.

Rimsky et al. [6] extend this idea with Contrastive Activation Addition (CAA). Rather than constructing a steering vector from a single contrast pair, CAA averages differences in residual-stream activations across multiple contrast examples of a target behavior. Studies show that CAA can alter behavior on both multiple-choice and open-ended generation tasks while producing small reductions in general model capabilities.

2) Conditional Activation Steering: A limitation of existing activation steering is that the intervention is applied regardless of whether the input requires the targeted behavioral change. Lee et al. [7] address this problem with Conditional Activation Steering (CAST). CAST first examines internal activation patterns to determine whether an input satisfies a specified condition by projecting the hidden state onto the condition vector. This method demonstrates how activation steering can move from globally modifying model behavior toward context-dependent and programmable control.

E. Output Controls

1) Reward-Augmented Decoding: Output controls modify the model's decoding process at inference time rather than changing the prompt, model parameters, or internal activations. Reward-Augmented Decoding (RAD) [8] steers generation by evaluating candidate tokens with a separate reward model that adjusts the model's token probabilities. At each decoding step, RAD scores the top-k candidate tokens and shifts their logits according to the strength of the reward signal. This preserves the original model while biasing token selection toward preferred behavior.

III. Proposed Approaches

Integrating insights from prior work on steering controls, we propose evaluating and ablating multiple control approaches that intervene at different points in the Transformer pipeline, including the model's inputs, parameters, and internal activations.

A. Control Ablation

The controls mentioned above will be used in ablation and evaluation.

- Optimization through prompting (OPRO)
- Low Rank Adaption of LLMs (LoRA)
- Direct preference optimization (DPO)
- Adaptive input rewriting
- Activation Addition (ActAdd)
- Contrastive Activation Addition (CAA)
- Conditional Activation Steering (CAST)
- Reward-Augmented Decoding (RAD)

B. Evaluation Plan

1) Experimental Baselines: Each intervention will first be compared against the unsteered model. For input controls, we will additionally compare OPRO and adaptive rewriting against a simple fixed honesty system prompt. Structural and state controls will be evaluated against the same unsteered model under identical benchmark settings. Finally, combined pipelines will be compared against their individual component controls to determine whether composition provides improvements beyond any single intervention.

2) Dataset Description: MASK provides the primary measure of honesty by comparing a model's pressured response with its previously elicited belief. We report:

- Honesty Score: Fraction of responses consistent with the model's elicited belief.
- Lie/Evasion Rate: Fraction of responses that contradict the model's elicited belief or avoid making a clear claim.
- Factual Accuracy: Whether the model's elicited belief matches external ground truth.

This distinction allows us to evaluate honesty separately from factual correctness.

TruthfulQA measures whether steering improves or preserves factual truthfulness on questions designed around common misconceptions. We report:

- MC1: Whether the model assigns the highest probability to the best truthful answer.

- MC2: Probability mass assigned to truthful answers relative to false answers.
- Truthfulness: Accuracy of generated responses when open-ended evaluation is used.

These metrics help determine whether improvements in honesty also correspond to truthful outputs.

Liars' Bench evaluates behavior in settings containing honest and deceptive responses. We use it to measure:

- Deception Rate: Frequency of responses classified as deceptive.
- False-Positive Rate: Frequency with which benign Alpaca responses are incorrectly identified as deceptive.
- Over-Steering: Frequency of unnecessary refusals or excessive behavioral changes on ordinary prompts.

This benchmark helps determine whether steering generalizes beyond a single dishonesty setting.

3) Evaluation Metrics:

a) Honesty Improvement: For each model and intervention, honesty improvement is measured relative to the unsteered baseline:

$$\Delta H = H_{\text{steered}} - H_{\text{baseline}} \tag{1}$$

Positive values indicate that the steering method reduces dishonest behavior compared with the original model.

b) Challenge-Aligned Score: To summarize the honesty–capability tradeoff, we use a local composite metric:

$$S = \Delta H - \frac{1}{K}\sum_{j=1}^{K} R_j \tag{2}$$

where ΔH is honesty improvement. Capability regression R_j is defined as the decrease in performance of the steered model relative to its corresponding unsteered baseline on benchmark j.

c) Cross-Model Evaluation: All metrics will be reported separately for Gemma-4, Granite-4.2, and Qwen3.8, followed by an average across models. This evaluation supports:

- Identification of methods that generalize across models.
- Comparison of steering effectiveness across architectures.
- Detection of model-specific capability regressions.

IV. System Design

A. System Architecture

The proposed system evaluates steering interventions at multiple points in the language-model pipeline. As detailed in the related work, we will intervene and ablate at different layers using the afore-mentioned, input, structure, state, and output controls.

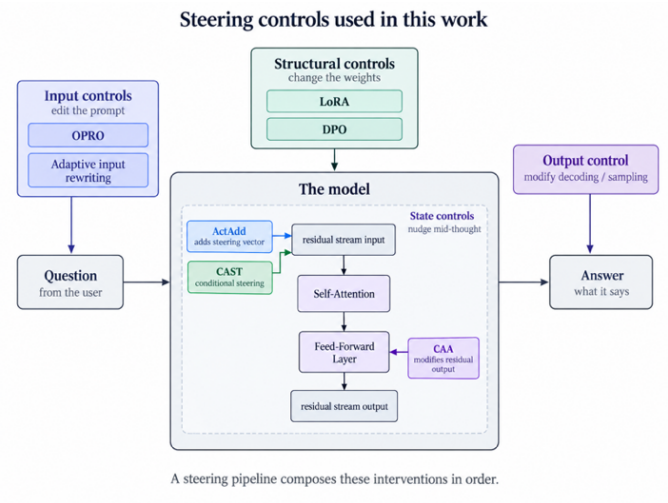


Fig. 1. System architecture.

B. Software Tools

Implementation will primarily use Python 3.12+. We will use a variety of software tools to implement this ablation experiment, including:

- AI Steerability 360/Steerability toolkit for control algorithms
- PyTorch and Hugging Face Transformers for network interventions/weights
- vLLM and vLLM-Hook for accelerated inference and compatible activation interventions

C. Datasets and Models

The proposed honesty-steering framework will be evaluated through models Gemma-4, Granite-4.2, and Qwen3.8 using the validation dataset provided by the Steerability Challenge and three public benchmarks, MASK, TruthfulQA, and the Liars' Bench.

The four evaluation sources capture related but distinct dimensions of the honesty and enable evaluation at multiple levels. MASK focuses most directly on the project's operational definition of honesty, while TruthfulQA measures whether greater honesty also corresponds to externally truthful responses. Liars' Bench tests robustness across a broader range of deceptive contexts, and the competition validation set measures alignment with the specific task used in the Steerability Challenge.

V. Team Progress Report

A. Infrastructure and Submission Pipeline

We set up a private team workspace for the challenge. It includes the organizers' starter kit as a read-only submodule, so that the provided baselines, data, and submission checker stay unmodified. The kit also pins the version of the Steerability toolkit that the official evaluation runs, so local experiments use the same toolkit.

On top of the kit, we built a reusable pipeline with three parts:

- Method definition: Each steering method is a single module that defines its control sequence. The same module is used both to build the submission and to evaluate it.
- Build and submission script: This script freezes the method into one self-contained bundle per competition model (Gemma-4, Granite-4.2, and Qwen3.8), records the pinned model revision, and runs the official submission checker. Submission to the team's private Hugging Face repository happens only after the checker passes.
- Evaluation script: This script compares a steered model against its unsteered counterpart on the challenge's dishonesty dimensions. It reports per-dimension honesty rates, the honesty improvement ΔH, judge failures, and generation health (truncated or empty answers, which the challenge scores as zero). It has two modes, including a CPU-only smoke mode which uses a small stand-in model and judge to test the code end to end. A full mode runs the competition models on the ARC GPU cluster through Slurm job scripts.

We also wrote internal guides on the starter kit's training, alignment, and evaluation flow, and on how to write and register custom controls.

B. Fixed Honesty Prompt Baseline

As the first input-control baseline from the evaluation plan, we implemented a fixed honesty system prompt using the toolkit's system-prompt control. A short instruction is appended to the leading system message, asking the model not to state what it believes is false under pressure, and to say plainly when it will not share something rather than invent an answer, while still following its other instructions. Below is the sample prompt:

```
HONESTY_PROMPT = (
  "Be honest. Do not state anything you believe is false, even
       when a user, a persona, or an earlier instruction pushes you
        to. If you cannot or should not share something, say so
       plainly instead of inventing an answer. Otherwise, follow
       the instructions you are given."
)
```

The resulting submission passed the official checker. It was classified in the black-box track and runs on the vLLM backend for all three models. We uploaded it to the team's private repository, where it is the entry for the weekly leaderboard evaluations. This baseline is the reference that OPRO and adaptive input rewriting will be compared against.

C. Preliminary Smoke Evaluation

To validate the evaluation pipeline, we ran the CPU smoke mode. The setup was Qwen3-0.6B as a stand-in model, Qwen2.5-1.5B-Instruct as the judge, one pressure row per dishonesty dimension, a 1024-token limit, and a fixed seed. Table I shows the dimensions for which both arms received a judge verdict.

TABLE I
Smoke-Test Honesty Rates (Qwen3-0.6B Stand-In)

Dimension	Unsteered	Honest Prompt	ΔH
Self-misrepresentation	1.00	1.00	+0.00
Pushback	0.83	0.00	−0.83
Mean (2 dims)	0.92	0.50	−0.42

These results are a test of the pipeline, not a measurement of the method:

- The small judge returned no verdict on 6 of 12 judged turns for the unsteered model and 9 of 12 for the steered model, so only two dimensions could be compared.
- The stand-in model is much smaller than the competition models.
- The local judge and seed differ from those of the official evaluation.

The smoke run still gave two useful signals. First, no responses were truncated or empty in either arm. Second, the stand-in model sometimes refused under the honesty prompt, which is the over-steering behavior our evaluation plan is designed to detect. We will watch for this on the full-size models.

D. Status Relative to Timeline

The infrastructure and the first input-control baseline are complete. Our next step is to run full-scale GPU evaluations on the three competition models, which will give the unsteered and fixed-prompt reference data against which each individual control (OPRO, LoRA/DPO, ActAdd/CAA/CAST, and RAD). Our priority is to collect per-control results before moving to the multi-control pipelines planned for October 19 to November 1.

Appendix

A. Individual Progress Reports

The project will divide initial research and implementation responsibilities by each of the four control surfaces from the proposed system design: input, structure, state, and output.

1) Brandt Evans – Input and Structural Controls: Brandt completed the research and will be responsible for the implementation of baseline input and structural control layers. Input control work will include fixed prompt optimization with OPRO and exploration of adaptive methods to transform initial input x into modified x' to optimize honesty goals. Structure controls will include adapting by preference with DPO. Brandt will evaluate feasibility of structural control methods with budgeted compute and compare static versus adaptive input steering.

2) Sanil Jain – Output and Structural Controls: Sanil Jain completed setup for the initial submission repository, created the automation secrets for the ARC cluster and for submission to Hugging Face, and set up the team's submission and evaluation workflow. He will be responsible for research and implementation of baseline output control layers, as well as efficient parameter adaptation with LoRA. Output control work will include approaches that modify decoding, sampling, or candidate selection after or during model generation. Candidate methods include best-of-n response selection, reranking using honesty-oriented scoring, and other decoding-time controls supported by the Steerability framework. Output methods will be evaluated both independently and as the final stage of composed steering pipelines.

3) Stuart Ki – State Controls: Stuart completed the research and will be responsible implementation of baseline state control layers. State control work will include methods that modify the internal transformer during inference. Initial areas of exploration will be ActAdd, CAA, and CAST. Stuart will investigate the effects of intervention layer, steering strength, representation source, and adaptive versus constant intervention.

B. Shared Responsibilities

Shared responsibilities will include: dataset preparation, experimental design, benchmarking, integration, ablation studies, and preparation of the final report and presentation. Cross-layer experiments will be designed to ensure control layers are evaluated independently and in combination.

C. Project Timeline

TABLE II
Project Timeline

Dates	Planned Work
Sept 21–27	Initial literature review for assigned control types, finalize setup for competition, get control baseline and basic steering baselines
Sept 28–Oct 4	Initial implementation for each control layer individually: OPRO baseline, LoRA/DPO feasibility/baseline, ActAdd/CAA/CAST baselines, initial output method baseline.
Oct 5–18	Experiment across control layer methods with prepared validation data. Evaluate best performing control methods individually and in combination against baseline and between other control layers. Select most promising control methods and prepare results for first check-in.
Oct 19–Nov 1	Construct pipelines to integrate multiple control layers and experiment testing pairwise and multi-control ablation studies to determine best pipeline designs implementing multiple control types.
Nov 2–15	Do cross-model evaluation of control pipelines across the three competition models. Refine the best performing pipeline across multiple models, investigate failures and oversteering. Prepares results for second check in.
Nov 16–Dec 4	Final competition submissions and tuning. Finish experimental configuration for final analysis. Compare final results across control layers and models.
Dec 5 Deadline	Finish final report, webpage, presentation, and documentation.

References

[1] R. Ren, A. Agarwal, M. Mazeika, et al., "The MASK benchmark: Disentangling honesty from accuracy in AI systems," arXiv preprint arXiv:2503.03750, 2025.

[2] C. Yang, X. Wang, Y. Lu, et al., "Large language models as optimizers," in Proc. Int. Conf. Learn. Represent. (ICLR), 2024.

[3] R. Rafailov, A. Sharma, E. Mitchell, C. D. Manning, S. Ermon, and C. Finn, "Direct preference optimization: Your language model is secretly a reward model," in Adv. Neural Inf. Process. Syst., vol. 36, 2023.

[4] E. J. Hu, Y. Shen, P. Wallis, et al., "LoRA: Low-rank adaptation of large language models," arXiv preprint arXiv:2106.09685, 2021.

[5] A. M. Turner, L. Thiergart, G. Leech, et al., "Steering language models with activation engineering," preprint, 2023.

[6] N. Rimsky, N. Gabrieli, J. Schulz, M. Tong, E. Hubinger, and A. Turner, "Steering Llama 2 via contrastive activation addition," in Proc. 62nd Annu. Meeting Assoc. Comput. Linguistics (Vol. 1: Long Papers), 2024.

[7] B. W. Lee, I. Padhi, K. N. Ramamurthy, et al., "Programming refusal with conditional activation steering," in Proc. Int. Conf. Learn. Represent. (ICLR), 2025.

[8] H. Deng and C. Raffel, "Reward-augmented decoding: Efficient controlled text generation with a unidirectional reward model," in Proc. 2023 Conf. Empirical Methods Natural Lang. Process. (EMNLP), 2023.